

Avalanche Dynamics in Software Dependency Networks

Avalanche Dynamics in Software Dependency Networks studies a stylized ecosystem of interconnected software libraries as a complex adaptive system. In the model, noise is a random developer update to an upstream library, connectivity is the dependency structure linking packages, and an avalanche is the number of downstream libraries that must update after compatibility buffers are exceeded.

The simulation uses 450 libraries, 5 replicate networks per connectivity regime, and 4500 recorded shocks after burn-in for each replicate. Across the sweep, mean in-degree rises from about 1.34 to 4.94 and the mean avalanche size grows by a factor of 3.23.

Large cascades also become more common: the probability of an avalanche of size at least 20 increases 1.46 percentage points between the lowest and highest connectivity conditions, and the largest observed cascade in the highest-connectivity regime reaches 424 libraries.

The results show a broad avalanche-size distribution and a persistent band of near-failure libraries, which is consistent with self-organized near-critical behaviour: the network is driven slowly by isolated updates and relaxes quickly through irregular cascades.

Model and Experimental Design

Why this is a good complexity-science system: software package ecosystems exhibit many interacting components, heterogeneity, path dependence, local compatibility rules, and global consequences that are not obvious from any single package.

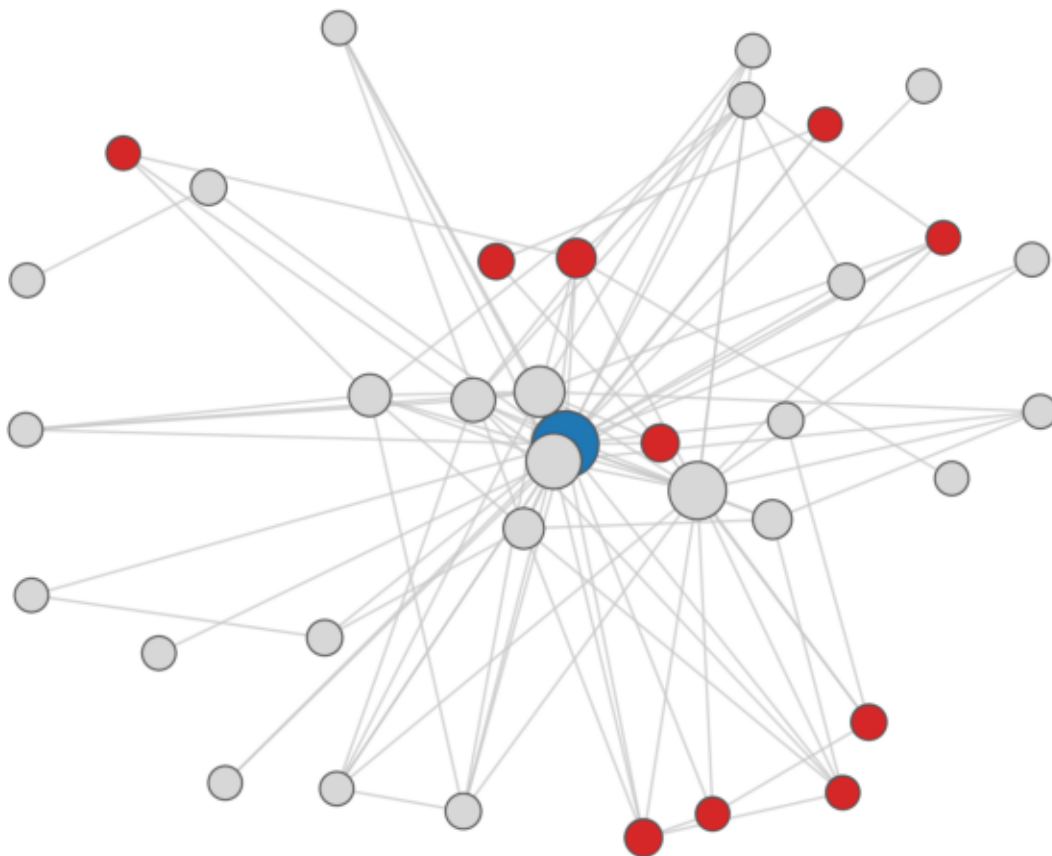
Model details: packages are added in age order to form a directed acyclic dependency graph. Preferential attachment makes popular libraries accumulate more dependents, producing hub-like structures similar to real package ecosystems. Every package stores the versions of its upstream dependencies that it last validated against.

Noise, avalanche, and connectivity mapping: a random source package receives a version increment of 1. A downstream package breaks when the accumulated upstream version drift reaches its compatibility buffer. When it breaks, it updates, resets its expected upstream versions, and can trigger further downstream failures. Connectivity is controlled by the target mean number of dependencies per package.

Analytical outputs: the code records avalanche size, maximum cascade depth, mean stress ratio, and the vulnerable fraction of libraries that are within one extra upstream version change of failure. These observables are used to compare connectivity regimes and to comment on self-organized criticality.

Figure 1. Example Update Cascade

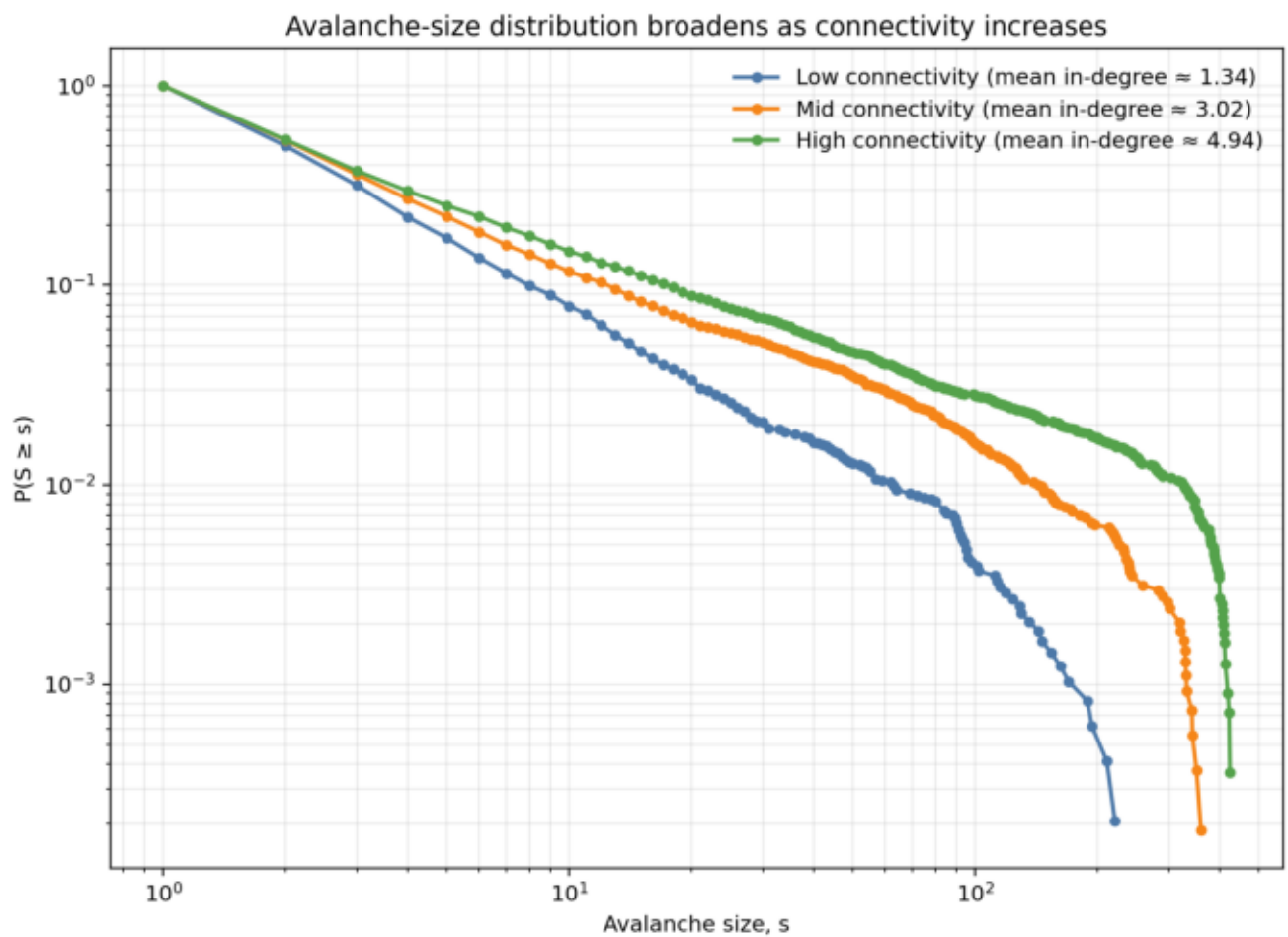
Illustrative cascade in a software dependency network



Blue: noisy source library 0 | Red: downstream libraries that failed and updated | Avalanche size = 10

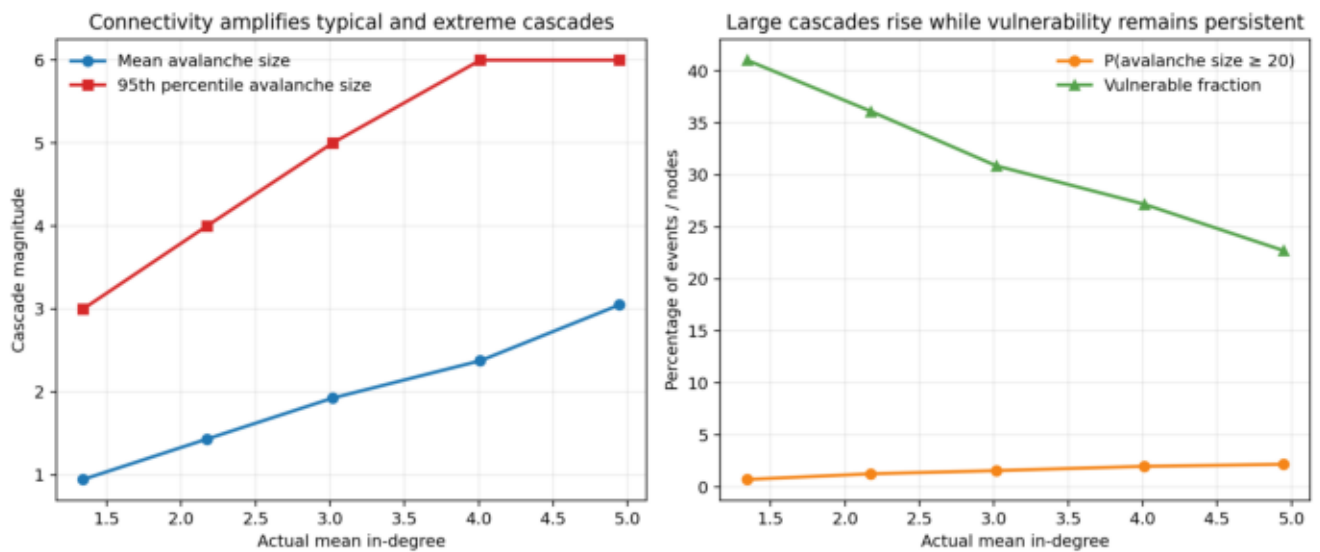
A single upstream update (blue node) can trigger a downstream avalanche (red nodes) because every library carries a finite compatibility buffer. This visual corresponds directly to the assignment definitions of noise, connectivity, and avalanche.

Figure 2. Avalanche-Size Distribution



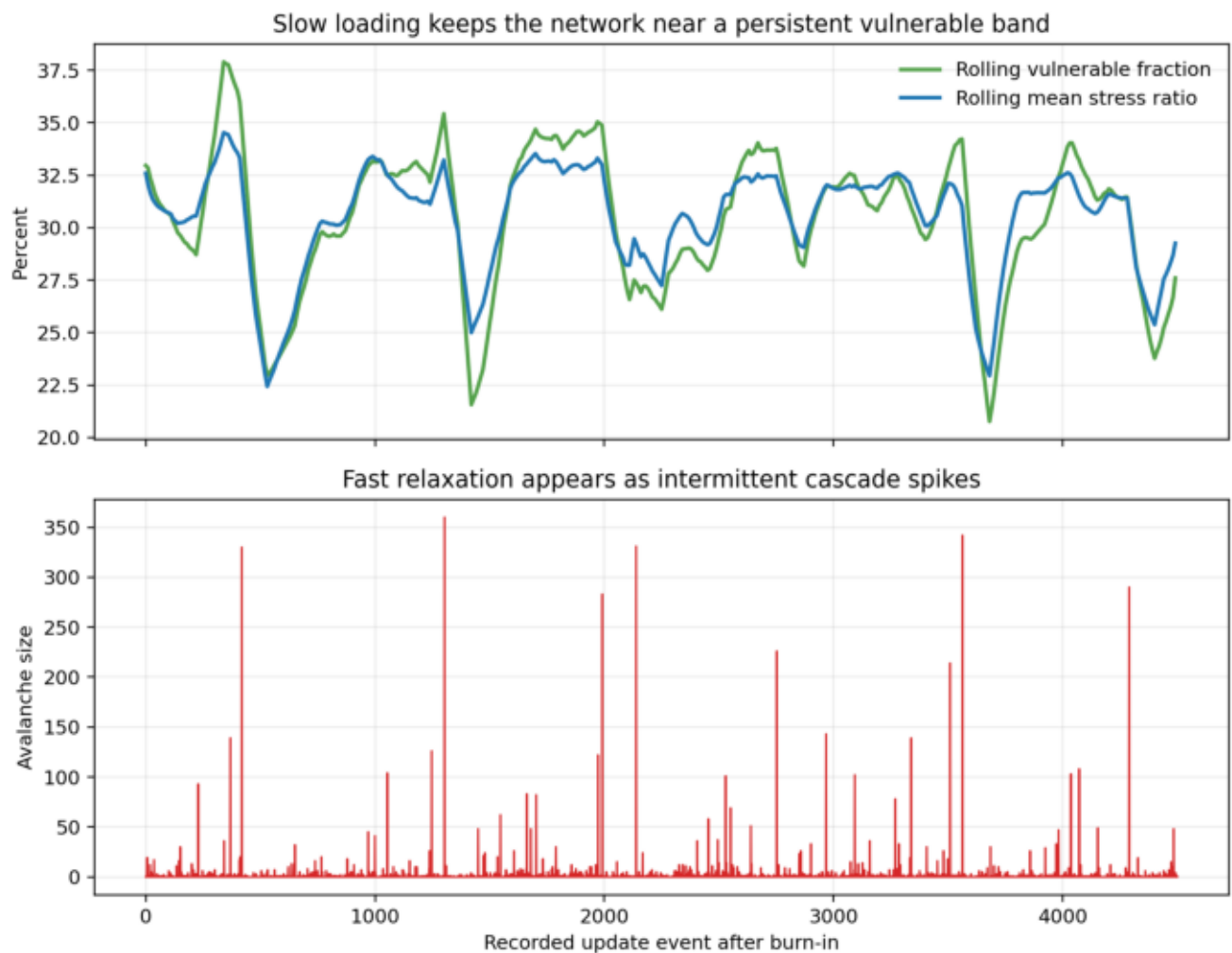
The complementary cumulative distribution broadens as connectivity rises, showing that denser dependency structures create a heavier tail of possible cascade sizes.

Figure 3. Connectivity and Cascade Risk



Increasing mean in-degree amplifies both the typical avalanche size and the probability of large cascades, but a persistent vulnerable fraction of libraries remains close to failure thresholds.

Figure 4. Slow Loading and Fast Relaxation



For a representative medium-connectivity network, the vulnerable fraction and stress ratio remain in a narrow band while avalanches appear as intermittent spikes, which is the characteristic signature of near-critical dynamics.

Results and Discussion

In the low-connectivity regime (mean in-degree about 1.34), the mean avalanche size is 0.94 and the 95th percentile is 3.00. Large cascades are relatively rare, with probability 0.72%.

In the middle regime (mean in-degree about 3.02), avalanches become broader and the network maintains a vulnerable fraction near 30.88%. This indicates that a substantial minority of libraries are continually poised close to their compatibility thresholds.

In the high-connectivity regime (mean in-degree about 4.94), the mean avalanche size rises to 3.05, the 95th percentile reaches 6.00, and the largest single cascade affects 424 libraries.

Taken together, the figures show the classic slow-drive and fast-relaxation pattern associated with critical systems: isolated updates steadily reload the network, while occasional cascades discharge the accumulated compatibility stress over many downstream packages.

Comment on self-organized criticality: this model does not prove exact critical exponents for a real software ecosystem, but it does display the main qualitative ingredients of self-organized critical behaviour. The system is driven slowly, relaxes rapidly, exhibits broad cascade distributions, and settles into a persistent near-threshold state without per-event retuning of parameters.

Interpretation: software dependency networks become more fragile as connectivity increases because each update has more possible downstream paths. At the same time, the network never becomes completely quiescent; instead it hovers in a regime where rare but very large cascades remain possible.

Limitations: real package ecosystems include semantic versioning, optional dependencies, maintainers, patch backports, and social coordination, none of which are modelled here. The simulation is therefore a stylized complexity-science analogue rather than a literal predictor of PyPI or npm behaviour.

Prompts used in preparing this project:

1. Create a complete complexity-science project on software dependency networks, using the assignment requirements and focusing strongly on the simulation and code.

2. Use software dependency networks as the topic, where a random update is the noise and downstream breakage cascades are the avalanches.

Generated locally from the included code and simulation outputs.